

E-Commerce Saga Pattern (Orchestration) - Flow Documentation

Services

1. Order Service
 2. Saga Service (Orchestrator)
 3. Inventory Service
 4. Payment Service (Next)
 5. Notification Service (Later)
-

Step 1 : Client Creates Order

API Call

POST /orders

Request

```
{ "orderId": "ORD1002", "product": "Laptop", "quantity": 1 }
```

Step 2 : Order Service Saves Order

Order Entity

Order order =

```
Order.builder().orderId(event.getOrderId()).product(event.getProduct()).quantity(event.getQuantity()).status("CREATED").build();
```

```
repository.save(order);
```

Database

orders

order_id	product	quantity	status
ORD1002	Laptop	1	CREATED

Step 3 : Create Outbox Event

```
OrderCreatedEvent event = new OrderCreatedEvent( UUID.randomUUID().toString(), order.getOrderid(),  
order.getProduct(), order.getQuantity() );
```

```
OutboxEvent outbox =  
OutboxEvent.builder() .eventType("ORDER_CREATED") .payload( mapper.writeValueAsString(event) ) .published(false) .createdAt
```

```
outboxRepository.save(outbox);
```

Database

outbox

id	event_type	published
1	ORDER_CREATED	false

Why Outbox?

Without Outbox:

Save Order → Kafka Down → Event Lost

Result:

Order exists Inventory never updated

System becomes inconsistent.

Outbox solves this problem.

Step 4 : Outbox Publisher Runs

Every 5 seconds

```
@Scheduled(fixedDelay = 5000)
```

```
List<OutboxEvent> events = repository.findByPublishedFalse();
```

For each event

```
producer.publish(event);
```

```
e.setPublished(true);
```

```
repository.save(e);
```

Step 5 : OrderCreatedEvent Published

Topic

order-created

Payload

OrderCreatedEvent

```
{ "eventId":"1111", "orderId":"ORD1002", "product":"Laptop", "quantity":1 }
```

Step 6 : Saga Service Receives Event

```
@KafkaListener( topics="order-created", groupId="saga-group" ) public void consume( OrderCreatedEvent event )
```

Create Saga

```
String sagaId = UUID.randomUUID().toString();
```

```
repository.save( new SagaState( sagaId, event.getOrderId(), "STARTED" ) );
```

Database

saga_state

saga_id	order_id	status
abc123	ORD1002	STARTED

Step 7 : Saga Sends Command

```
ReserveInventoryCommand command = new ReserveInventoryCommand( sagaId, event.getOrderId(),  
event.getProduct(), event.getQuantity() );
```

```
producer.sendReserveInventory(command);
```

Topic

reserve-inventory

Step 8 : Inventory Service Receives Command

```
@KafkaListener( topics="reserve-inventory", groupId="inventory-group" )
```

```
public void consume( ReserveInventoryCommand command )
```

Check duplicate

```
if(repository.existsById(command.getSagaId())) { return; }
```

Step 9 : Reduce Stock

```
Inventory inventory =  
repository.findById( command.getProduct() ).orElse( Inventory.builder() .product(command.getProduct()) .stock(100) .build() );
```

```
inventory.setStock( inventory.getStock() - command.getQuantity() );
```

```
repository.save(inventory);
```

Database

inventory

product	stock
Laptop	99

Step 10 : Publish Success Event

```
InventoryReservedEvent event = new InventoryReservedEvent( command.getSagaId(),  
command.getOrderId(), command.getProduct(), command.getQuantity() );
```

```
producer.publishReserved(event);
```

Topic

inventory-reserved

Step 11 : Saga Receives Success Event

```
@KafkaListener( topics="inventory-reserved", groupId="saga-group" )
```

```
public void consume( InventoryReservedEvent event )
```

Saga status update

```
saga.setStatus( "INVENTORY_RESERVED" );
```

```
repository.save(saga);
```

Step 12 : Saga Sends Payment Command

```
ProcessPaymentCommand command = new ProcessPaymentCommand( event.getSagaId(),  
event.getOrderId() );
```

```
producer.sendPayment(command);
```

Topic

process-payment

Current Flow Completed

Order Service | | OrderCreatedEvent v Saga Service | | ReserveInventoryCommand v Inventory Service | |
InventoryReservedEvent v Saga Service | | ProcessPaymentCommand v Payment Service (Next)

What We Build Next

Payment Service

Consumes

ProcessPaymentCommand

Produces

PaymentCompletedEvent

OR

PaymentFailedEvent

After Payment Success

Payment Service | | PaymentCompletedEvent v Saga Service | | OrderApprovedCommand v Order Service

Order Status

CREATED → APPROVED

Saga Status

PAYMENT_COMPLETED

SAGA_COMPLETED

Failure Path

Inventory Failed

InventoryFailedEvent | v Saga Service | v Order Rejected

Status

REJECTED

Payment Failed

PaymentFailedEvent | v Saga Service | v Release Inventory | v Inventory Released Event | v Order Rejected

This is called a Compensation Transaction.